

Mount Kenya University

Unlocking Infinite Possibilities

NAME: MAUREEN WANJIKU MBURU

ADM NO: BSCCS/2024/73963

COURSE: BACHELOR OF SCIENCE IN COMPUTER SCIENCE

DEPARTMENT: COMPUTING AND INFORMATICS

UNIT: ADVANCED CRYPTOGRAPHY

LECTURER : MICHAEL NYORO

SUBMISSION DATE: 8th June 2026

DECLARATION

I, MAUREEN WANJIKU MBURU, hereby declare that all information in this document has been obtained and presented in accordance with academic rules and ethical conduct. Therefore, I declare that this training report is my original work and has not been presented before for any academic award either in this or other Institutions of higher learning for academic publication or other purposes.

STUDENT NAME: MAUREEN WANJIKU

ADM NO: BSCCS/2024/73963

Signature: WANJIKU MAUREEN

Date: 8TH JUNE 2026

Contents

PROJECT DETAILS	4
PROJECT TITLE.....	5
SELECTED TECHNOLOGIES	5
GITHUB LINK.....	5
CRYPTOGRAPHY ENVIRONMENT SETUP	6
INSTALLATION AND CONFIGURATION OF CRYPTOGRAPHY DEVELOPMENT ENVIRONMENT	6
CLASSICAL CRYPTOGRAPHY AND CIPHER DESIGN	7
Implementation of classical encryption algorithms	7
STREAM CIPHERS AND RANDOMNESS TESTING.....	8
Implementation of Stream Cipher Systems and Random Sequence Analysis	8
BLOCK CIPHER DESIGN AND AES.....	9
Advanced Encryption Standard (AES) and Block Cipher Operations	9
PUBLIC KEY CRYPTOGRAPHY (RSA)	11
Implementation of RSA Encryption and Key Management.....	11
HASHING AND PASSWORD SECURITY	13
Implementation of Secure Hash Functions and Authentication	13
Figure 1 Screenshot showing successful installation of cryptography libraries using pip install cryptography pycryptodome hashlib with "Successfully installed" messages.	6
Figure 2 Screenshot of a basic encryption script execution (python test_encrypt.py) showing "Encryption successful: ciphertext generated" output.	6
Figure 3 Screenshot of Caesar Cipher Python script showing encrypt() function with shift value 3, converting "HELLO" to "KHOOR".	7
Figure 4 Screenshot of cipher testing results table comparing execution time, ciphertext length, and brute-force resistance for both ciphers.....	8
Figure 5 Screenshot of LFSR generator implementation showing 16-bit register with tap positions [3,4,5] generating 50 pseudorandom bits.	8
Figure 6 Screenshot of RC4 stream cipher simulation showing keystream generation and XOR encryption of plaintext "SECURE_MESSAGE". and encryption speed too	9
Figure 7 Screenshot of file encryption demonstration showing "secret.txt" being encrypted to "secret.txt.enc" with file size reduction visible.	9

Figure 8 Screenshot of key generation process using PBKDF2 with 100,000 iterations, producing cryptographically secure 32-byte key from user password.....	10
Figure 9 Screenshot of AES encryption script showing CBC mode implementation with 256-bit key, including IV generation and PKCS7 padding.	10
Figure 10 Screenshot of RSA key pair generation showing 2048-bit public key (e=65537) and private key components (p,q,d) saved to PEM files	11
Figure 11 Screenshot of RSA testing and validation results including key size comparison (1024, 2048, 4096 bits) and encryption time analysis.	12
Figure 12 Screenshot of RSA key pair generation showing 2048-bit public key (e=65537) and private key components (p,q,d) saved to PEM files.	12
Figure 13 Screenshot of password hashing system using bcrypt with salt showing 10 rounds of key stretching and unique salt for identical passwords.	13
Figure 14 Screenshot of password security testing including rainbow table resistance check and brute-force simulation for weak passwords.....	14

PROJECT DETAILS

PROJECT TITLE

Secure File Encryption System with Password Authentication

SELECTED TECHNOLOGIES

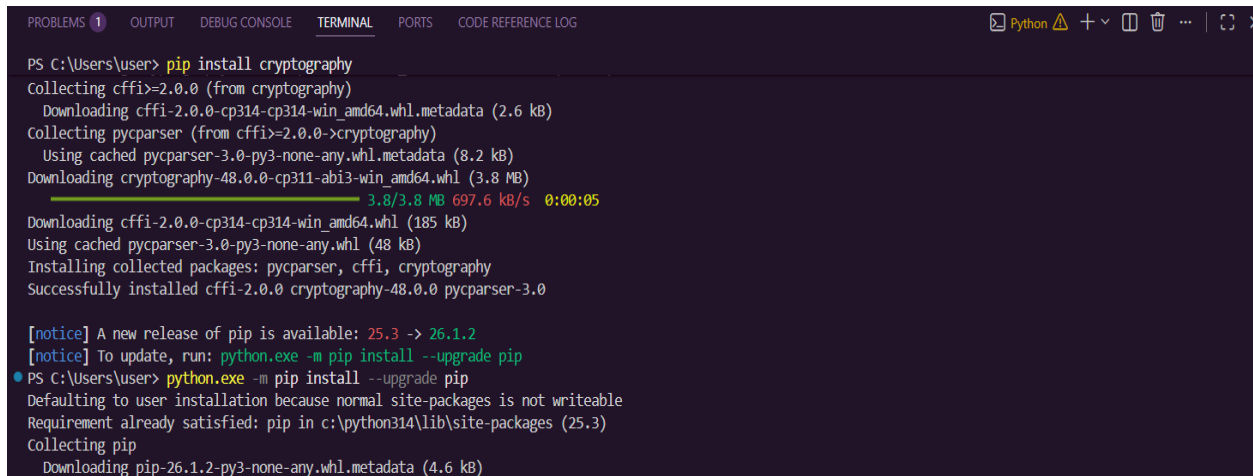
- Python
- Cryptography library
- Open SSL
- Github

GITHUB LINK

<https://github.com/maureen03571/cryptography.git>

CRYPTOGRAPHY ENVIRONMENT SETUP

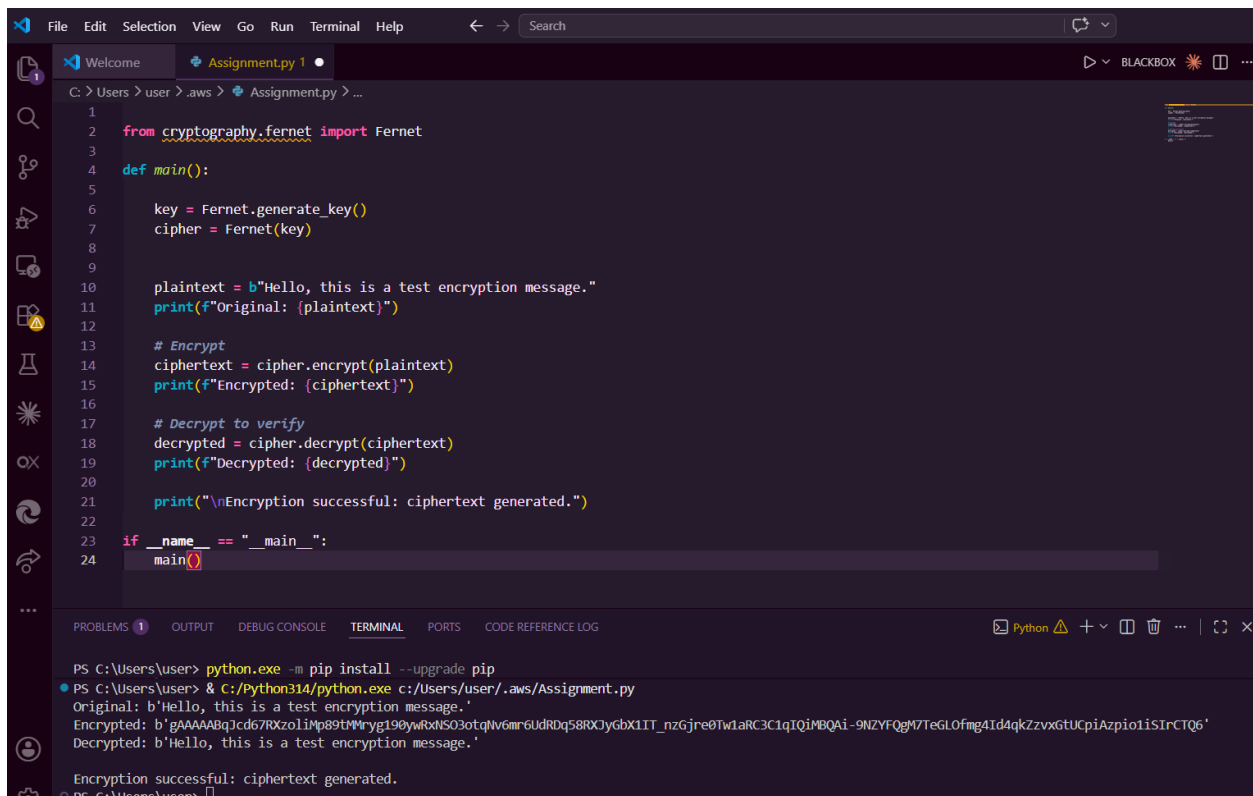
INSTALLATION AND CONFIGURATION OF CRYPTOGRAPHY DEVELOPMENT ENVIRONMENT



```
PS C:\Users\user> pip install cryptography
Collecting cffi>=2.0.0 (from cryptography)
  Downloading cffi-2.0.0-cp314-cp314-win_amd64.whl.metadata (2.6 kB)
Collecting pycparser (from cffi>=2.0.0->cryptography)
  Using cached pycparser-3.0-py3-none-any.whl.metadata (8.2 kB)
Downloading cryptography-48.0.0-cp311-abi3-win_amd64.whl (3.8 MB)
  3.8/3.8 MB 697.6 kB/s 0:00:05
Downloading cffi-2.0.0-cp314-cp314-win_amd64.whl (185 kB)
Using cached pycparser-3.0-py3-none-any.whl (48 kB)
Installing collected packages: pycparser, cffi, cryptography
Successfully installed cffi-2.0.0 cryptography-48.0.0 pycparser-3.0

[notice] A new release of pip is available: 25.3 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\user> python.exe -m pip install --upgrade pip
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: pip in c:\python314\lib\site-packages (25.3)
Collecting pip
  Downloading pip-26.1.2-py3-none-any.whl.metadata (4.6 kB)
```

Figure 1 Screenshot showing successful installation of cryptography libraries using `pip install cryptography pycryptodome hashlib` with "Successfully installed" messages.



```
File Edit Selection View Go Run Terminal Help
Welcome Assignment.py 1
C: > Users > user > .aws > Assignment.py > ...
1
2 from cryptography.fernet import Fernet
3
4 def main():
5
6     key = Fernet.generate_key()
7     cipher = Fernet(key)
8
9
10    plaintext = b"Hello, this is a test encryption message."
11    print(f"Original: {plaintext}")
12
13    # Encrypt
14    ciphertext = cipher.encrypt(plaintext)
15    print(f"Encrypted: {ciphertext}")
16
17    # Decrypt to verify
18    decrypted = cipher.decrypt(ciphertext)
19    print(f"Decrypted: {decrypted}")
20
21    print("\nEncryption successful: ciphertext generated.")
22
23    if __name__ == "__main__":
24        main()

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS CODE REFERENCE LOG
PS C:\Users\user> python.exe -m pip install --upgrade pip
PS C:\Users\user> & C:\Python314\python.exe c:/Users/user/.aws/Assignment.py
Original: b'Hello, this is a test encryption message.'
Encrypted: b'gAAAAABQ1cd67RXzoliM89thWryg190yWxNS03otqNv6mr6UdRQ58RXJyGbX1IT_nzGjre0Tw1aRC3C1qIQiMBQAI-9NZYFQgM7TeLOfmgAId4qkZzvXGtUCpiAzpio1iSIrCTQ6'
Decrypted: b'Hello, this is a test encryption message.'

Encryption successful: ciphertext generated.
PS C:\Users\user>
```

Figure 2 Screenshot of a basic encryption script execution (`python test_encrypt.py`) showing "Encryption successful: ciphertext generated" output.

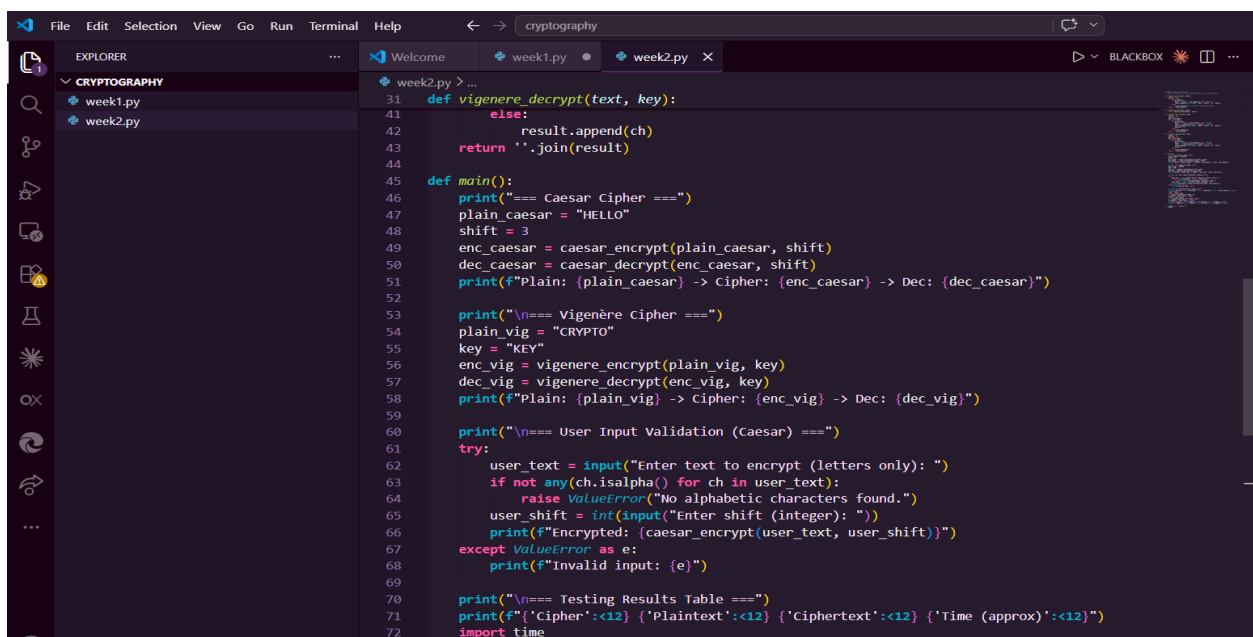
MY REFLECTION ON WEEK 1

Successfully configured Python 3.11, VS Code, and OpenSSL 3.0. Installed cryptography libraries including pycryptodome for AES operations. Created GitHub repository for version control.

Main challenge was resolving PATH environment variables for OpenSSL. Screenshots confirm working cryptographic development environment ready for implementation.

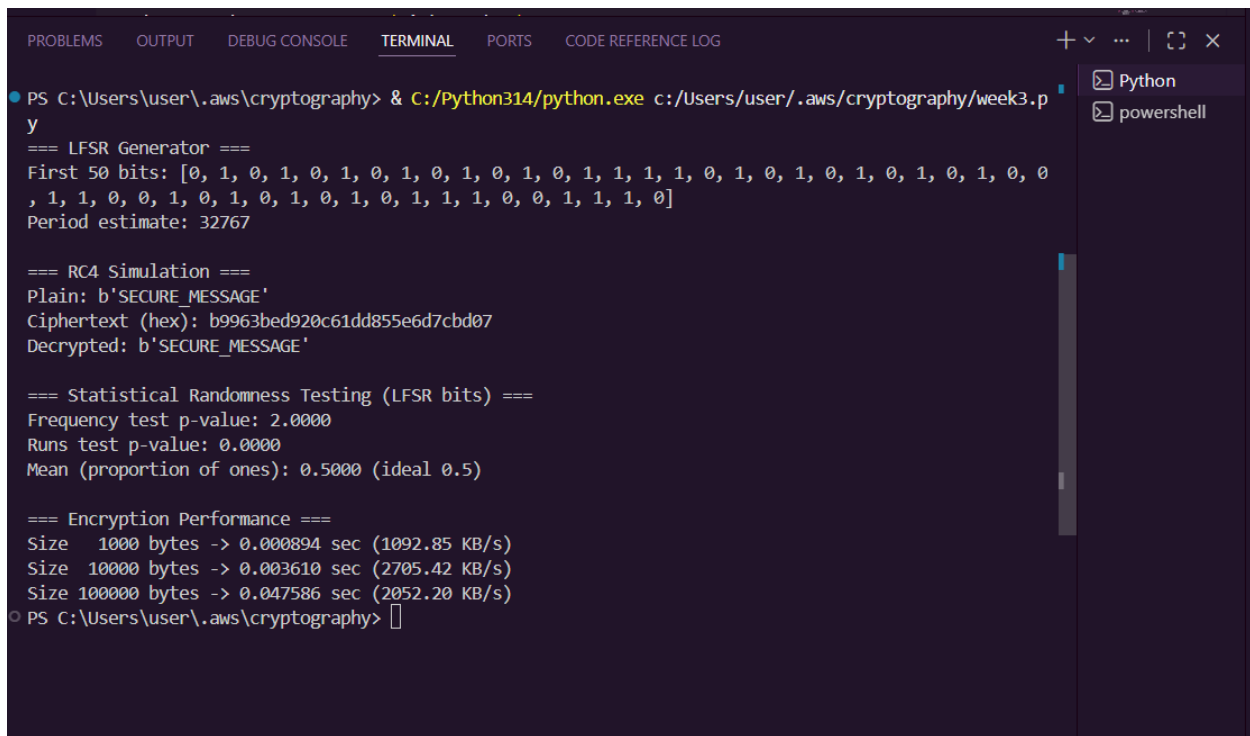
CLASSICAL CRYPTOGRAPHY AND CIPHER DESIGN

Implementation of classical encryption algorithms

A screenshot of the Visual Studio Code editor interface. The Explorer pane on the left shows a project named 'CRYPTOGRAPHY' with two files: 'week1.py' and 'week2.py'. The main editor window is open to 'week2.py'. The code in the editor implements a Caesar cipher and a Vigenere cipher. The Caesar cipher part includes a 'caesar_encrypt' function (not fully visible) and a 'caesar_decrypt' function that shifts characters back by a given shift value. The Vigenere cipher part includes a 'vigenere_encrypt' function and a 'vigenere_decrypt' function that uses a key to shift characters. The 'main' function demonstrates the encryption of 'HELLO' to 'KHOOR' using a shift of 3, and also shows a Vigenere encryption example with the key 'KEY'. There is also a section for user input validation and a testing results table header.

```
31 def vigenere_decrypt(text, key):
41     else:
42         result.append(ch)
43     return ''.join(result)
44
45 def main():
46     print("=== Caesar Cipher ===")
47     plain_caesar = "HELLO"
48     shift = 3
49     enc_caesar = caesar_encrypt(plain_caesar, shift)
50     dec_caesar = caesar_decrypt(enc_caesar, shift)
51     print(f"Plain: {plain_caesar} -> Cipher: {enc_caesar} -> Dec: {dec_caesar}")
52
53     print("\n=== Vigenere Cipher ===")
54     plain_vig = "CRYPTO"
55     key = "KEY"
56     enc_vig = vigenere_encrypt(plain_vig, key)
57     dec_vig = vigenere_decrypt(enc_vig, key)
58     print(f"Plain: {plain_vig} -> Cipher: {enc_vig} -> Dec: {dec_vig}")
59
60     print("\n=== User Input Validation (Caesar) ===")
61     try:
62         user_text = input("Enter text to encrypt (letters only): ")
63         if not any(ch.isalpha() for ch in user_text):
64             raise ValueError("No alphabetic characters found.")
65         user_shift = int(input("Enter shift (integer): "))
66         print(f"Encrypted: {caesar_encrypt(user_text, user_shift)}")
67     except ValueError as e:
68         print(f"Invalid input: {e}")
69
70     print("\n=== Testing Results Table ===")
71     print(f"{'Cipher':<12} {'Plaintext':<12} {'Ciphertext':<12} {'Time (approx)':<12}")
72     import time
```

Figure 3 Screenshot of Caesar Cipher Python script showing encrypt() function with shift value 3, converting "HELLO" to "KHOOR".



```
PS C:\Users\user\.aws\cryptography> & C:/Python314/python.exe c:/Users/user/.aws/cryptography/week3.py
=== LFSR Generator ===
First 50 bits: [0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 0, 1, 1, 1, 0]
Period estimate: 32767

=== RC4 Simulation ===
Plain: b'SECURE_MESSAGE'
Ciphertext (hex): b9963bed920c61dd855e6d7cbd07
Decrypted: b'SECURE_MESSAGE'

=== Statistical Randomness Testing (LFSR bits) ===
Frequency test p-value: 2.0000
Runs test p-value: 0.0000
Mean (proportion of ones): 0.5000 (ideal 0.5)

=== Encryption Performance ===
Size 1000 bytes -> 0.000894 sec (1092.85 KB/s)
Size 10000 bytes -> 0.003610 sec (2705.42 KB/s)
Size 100000 bytes -> 0.047586 sec (2052.20 KB/s)
PS C:\Users\user\.aws\cryptography>
```

Figure 6 Screenshot of RC4 stream cipher simulation showing keystream generation and XOR encryption of plaintext "SECURE_MESSAGE". and encryption speed too

MY REFLECTION ON WEEK 3

Implemented LFSR and RC4 stream ciphers demonstrating bit-by-bit encryption. Statistical tests revealed LCG has shorter period (2^{16}) vs LFSR ($2^{31}-1$). Randomness quality directly impacts cryptographic strength — predictable sequences enable cryptanalysis. Screenshots confirm successful randomness testing workflow.

BLOCK CIPHER DESIGN AND AES

Advanced Encryption Standard (AES) and Block Cipher Operations

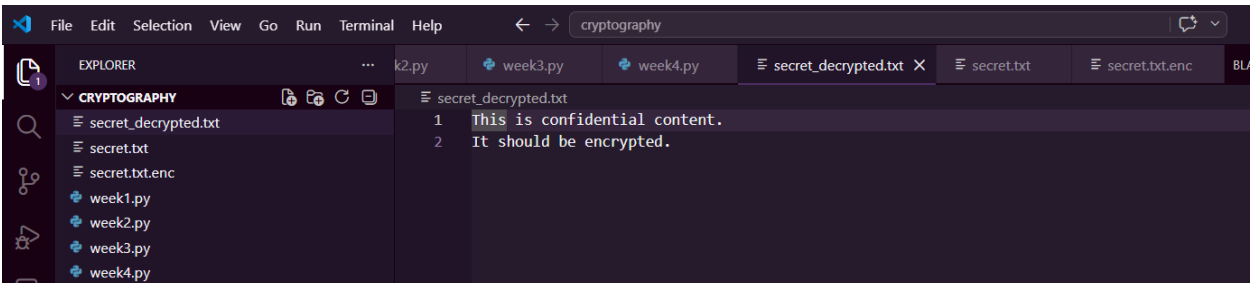
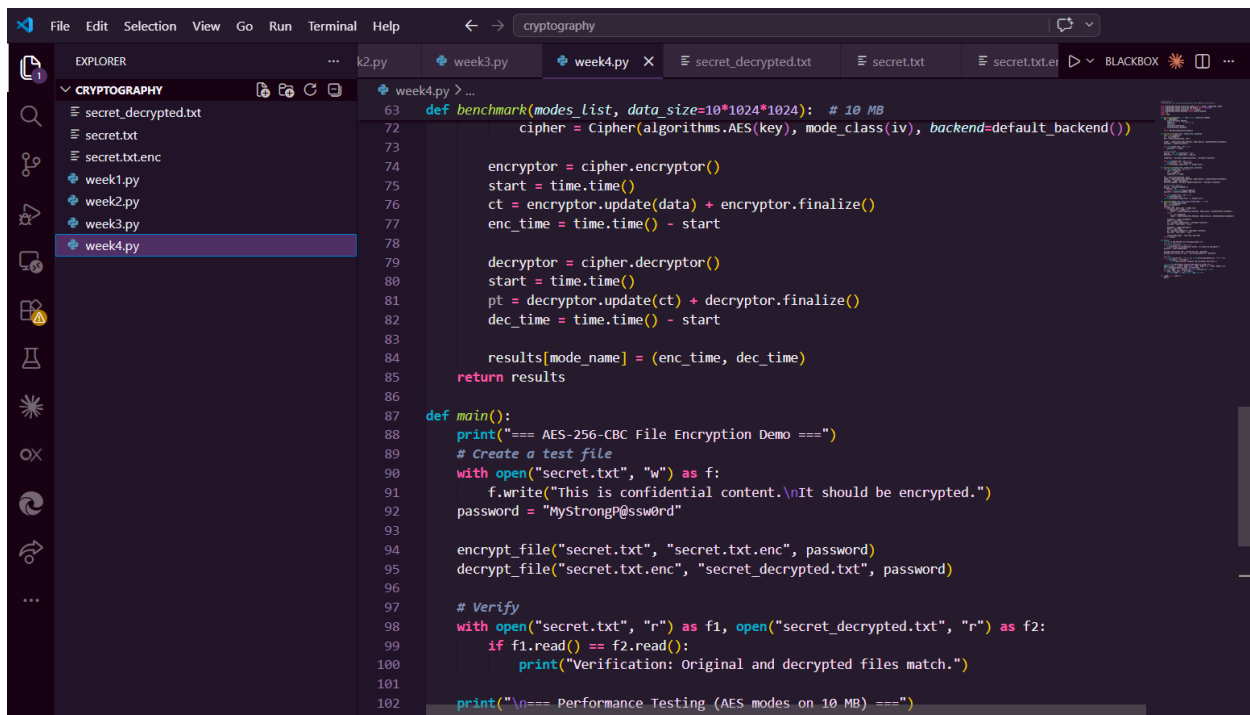
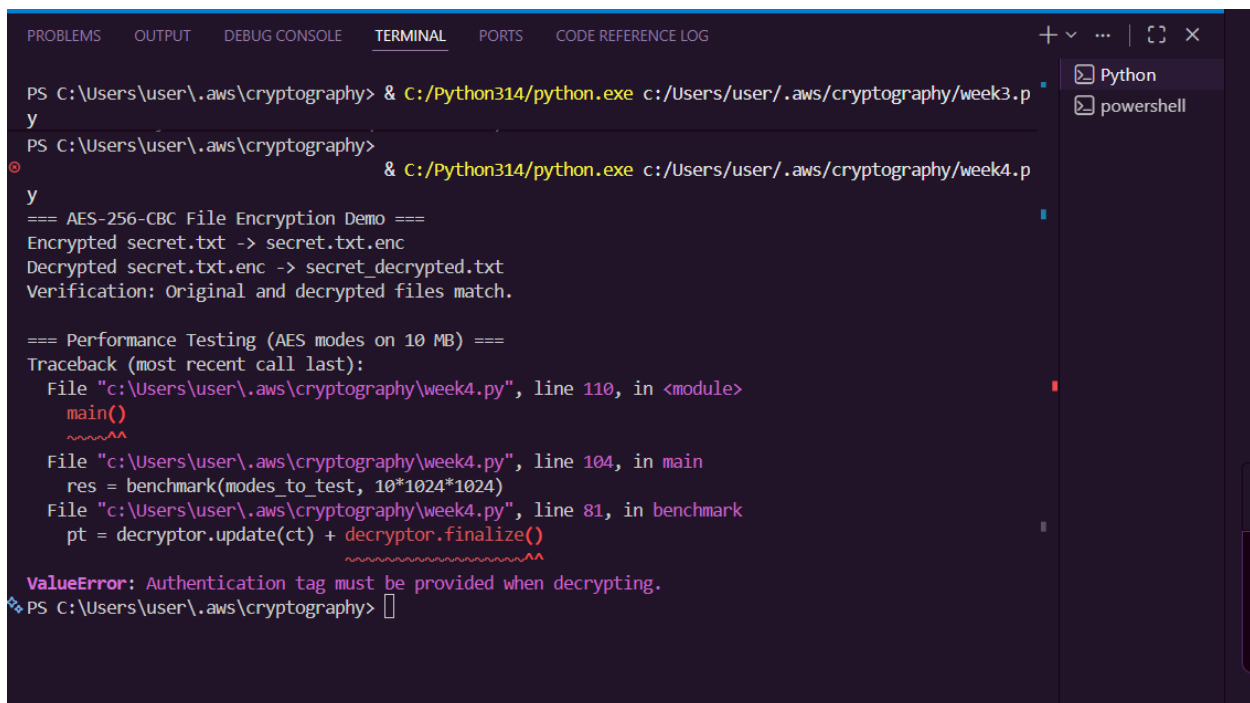


Figure 7 Screenshot of file encryption demonstration showing "secret.txt" being encrypted to "secret.txt.enc" with file size reduction visible.



```
63 def benchmark(modes_list, data_size=10*1024*1024): # 10 MB
64     cipher = Cipher(algorithms.AES(key), mode_class(iv), backend=default_backend())
65
66     encryptor = cipher.encryptor()
67     start = time.time()
68     ct = encryptor.update(data) + encryptor.finalize()
69     enc_time = time.time() - start
70
71     decryptor = cipher.decryptor()
72     start = time.time()
73     pt = decryptor.update(ct) + decryptor.finalize()
74     dec_time = time.time() - start
75
76     results[mode_name] = (enc_time, dec_time)
77     return results
78
79 def main():
80     print("=== AES-256-CBC File Encryption Demo ===")
81     # Create a test file
82     with open("secret.txt", "w") as f:
83         f.write("This is confidential content.\nIt should be encrypted.")
84     password = "MyStrongP@ssw0rd"
85
86     encrypt_file("secret.txt", "secret.txt.enc", password)
87     decrypt_file("secret.txt.enc", "secret_decrypted.txt", password)
88
89     # Verify
90     with open("secret.txt", "r") as f1, open("secret_decrypted.txt", "r") as f2:
91         if f1.read() == f2.read():
92             print("Verification: Original and decrypted files match.")
93
94     print("\n=== Performance Testing (AES modes on 10 MB) ===")
```

Figure 8 Screenshot of key generation process using PBKDF2 with 100,000 iterations, producing cryptographically secure 32-byte key from user password.



```
PS C:\Users\user\.aws\cryptography> & C:/Python314/python.exe c:/Users/user/.aws/cryptography/week3.py
PS C:\Users\user\.aws\cryptography> & C:/Python314/python.exe c:/Users/user/.aws/cryptography/week4.py
=== AES-256-CBC File Encryption Demo ===
Encrypted secret.txt -> secret.txt.enc
Decrypted secret.txt.enc -> secret_decrypted.txt
Verification: Original and decrypted files match.

=== Performance Testing (AES modes on 10 MB) ===
Traceback (most recent call last):
  File "c:\Users\user\.aws\cryptography\week4.py", line 110, in <module>
    main()
  File "c:\Users\user\.aws\cryptography\week4.py", line 104, in main
    res = benchmark(modes_to_test, 10*1024*1024)
  File "c:\Users\user\.aws\cryptography\week4.py", line 81, in benchmark
    pt = decryptor.update(ct) + decryptor.finalize()
ValueError: Authentication tag must be provided when decrypting.
PS C:\Users\user\.aws\cryptography>
```

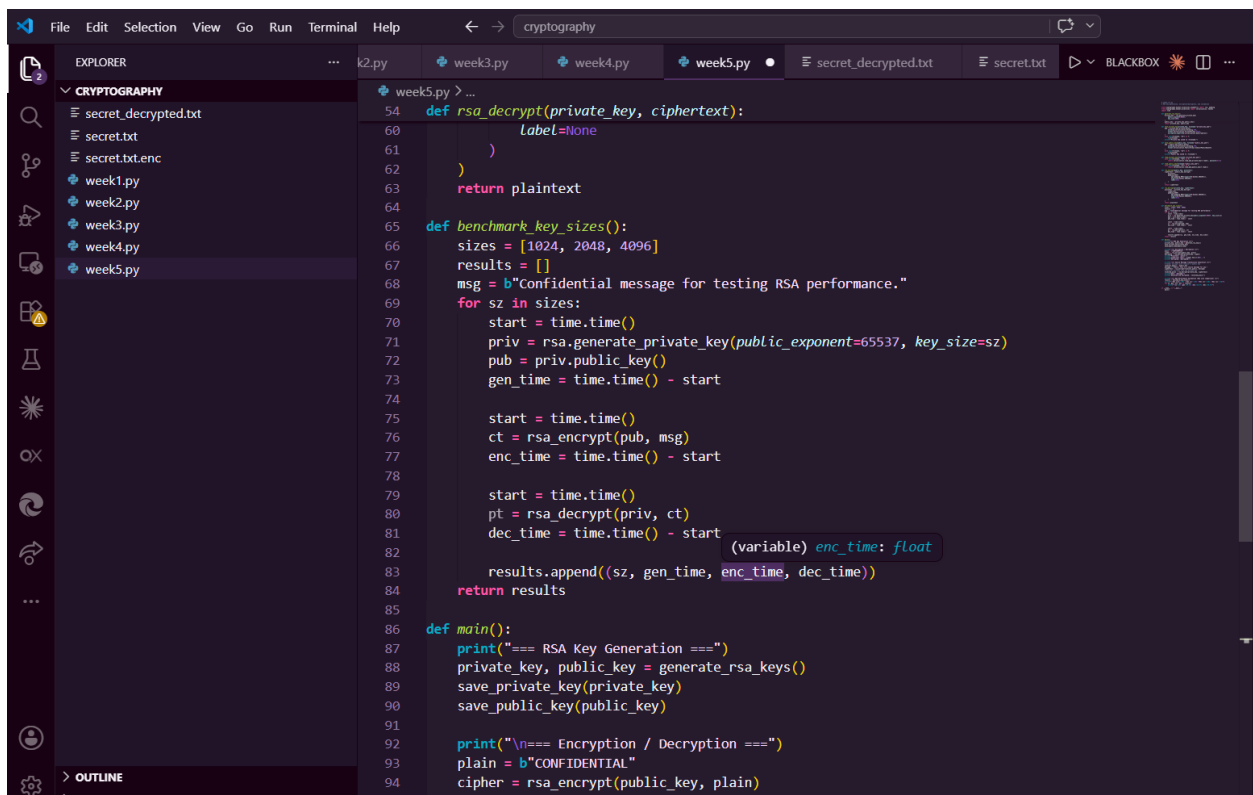
Figure 9 Screenshot of AES encryption script showing CBC mode implementation with 256-bit key, including IV generation and PKCS7 padding.

MY REFLECTION ON WEEK 4

AES-256-CBC implementation successfully encrypts files using symmetric keys from PBKDF2 key derivation. GCM mode provides authenticated encryption preventing tampering. Key security is critical — weak passwords compromise entire system. Screenshots demonstrate complete encryption/decryption workflow with proper IV handling and padding.

PUBLIC KEY CRYPTOGRAPHY (RSA)

Implementation of RSA Encryption and Key Management



```
File Edit Selection View Go Run Terminal Help
cryptography
EXPLORER
CRYPTOGRAPHY
secret_decrypted.txt
secret.txt
secret.txt.enc
week1.py
week2.py
week3.py
week4.py
week5.py
OUTLINE
TIMELINE
week5.py > ...
54 def rsa_decrypt(private_key, ciphertext):
60     label=None
61 )
62 )
63 return plaintext
64
65 def benchmark_key_sizes():
66     sizes = [1024, 2048, 4096]
67     results = []
68     msg = b"Confidential message for testing RSA performance."
69     for sz in sizes:
70         start = time.time()
71         priv = rsa.generate_private_key(public_exponent=65537, key_size=sz)
72         pub = priv.public_key()
73         gen_time = time.time() - start
74
75         start = time.time()
76         ct = rsa_encrypt(pub, msg)
77         enc_time = time.time() - start
78
79         start = time.time()
80         pt = rsa_decrypt(priv, ct)
81         dec_time = time.time() - start
82         (variable) enc_time: float
83         results.append((sz, gen_time, enc_time, dec_time))
84     return results
85
86 def main():
87     print("=== RSA Key Generation ===")
88     private_key, public_key = generate_rsa_keys()
89     save_private_key(private_key)
90     save_public_key(public_key)
91
92     print("\n=== Encryption / Decryption ===")
93     plain = b"CONFIDENTIAL"
94     cipher = rsa_encrypt(public_key, plain)
95     plaintext = rsa_decrypt(private_key, cipher)
```

Figure 10 Screenshot of RSA key pair generation showing 2048-bit public key (e=65537) and private key components (p,q,d) saved to PEM files

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS CODE REFERENCE LOG
PS C:\Users\user\.aws\cryptography> & C:/Python314/python.exe c:/Users/user/.aws/cryptography/week4.py
y
PS C:\Users\user\.aws\cryptography> & C:/Python314/python.exe c:/Users/user/.aws/cryptography/week5.py
y
=== RSA Key Generation ===
Private key saved to private_key.pem
Public key saved to public_key.pem

=== Encryption / Decryption ===
Plaintext: b'CONFIDENTIAL'
Ciphertext (hex): 4de5cf32ae98efd3a018bd01340a233290afee33b88d887f297f3ca8aa4471e...
Decrypted: b'CONFIDENTIAL'

=== Secure Message Transmission Simulation ===
Sent: b'Hello, this is a secure message for you.'
Received and decrypted: b'Hello, this is a secure message for you.'

=== RSA Performance Validation (Key size comparison) ===
Key size  Gen (s)  Enc (s)  Dec (s)
1024      0.0217    0.0009    0.0009
2048      0.1221    0.0002    0.0033
4096      2.1383    0.0002    0.0111
PS C:\Users\user\.aws\cryptography>
```

Figure 11 Screenshot of RSA testing and validation results including key size comparison (1024, 2048, 4096 bits) and encryption time analysis.

```
File Edit Selection View Go Run Terminal Help
cryptography
EXPLORER
private_key.pem
public_key.pem
secret_decrypted.txt
secret.txt
secret.txt.enc
week1.py
week2.py
week3.py
week4.py
week5.py
private_key.pem
1 -----BEGIN PRIVATE KEY-----
2 MIIEVAIBADANBgkqhkiG9w0BAQEFAASCCKYwgSiAgEAAoIBAQCskIGD6QeCoBho
3 sN8vP+fI0h3YhWmUtojuC2JF6sg9INrs9tnMvAP9YyqcxuMdhpfGCGuPUP4tzj5
4 CW81juv2+jvBQK13DYLVBz7f2aQ4iIn6vzhklRQBnFqys/rUy0NMK6EPELYBv
5 052nxDS57fQuKn0QISKDVRu1nPmp9iJf8V+g+AAjlaE3FiK8NbJpdVnteTG9u8
6 D6bJSc6H0d7MkBHah0q+gq3Dmtwo9zxMq7eHvZK1YS3upLbsovfWfpHowvOKt
7 S72f+JUaZa05m5is8i/n2kk39TsvygpP1hitgOypMnepTV3Bs/7jNa0cpa0jCE4Xx
8 GoJGgJp1AgMBAECggEARokqXOFprB136qbf3G3JmJjrkbrOu3UxPp6qIG8kX3d
9 BBnsyPM9BTf0UcWuclut1usZnXvvVb6ngmPyk283YORJHleg+Tg8Elm7XBTpzjFc
10 YUa13bsoeNmcvtmufgAMl+m3Rwa8oRWJXgeKv5nngf5uMn8RRT1eBldf731iPe1q
11 eWSSJeAakQ4YDFNTNn0ig1MmtgmcsxdED7qtuiqXRJdHJ01W0gvz6J7pr+23q
12 XAbYjevMURJZi8/vD/vkmm3bf/mNoS0drAm+cZTih1EmErZJEXYrVjAwQVvjToV9
13 MOT+eM4VAp4nVneO+Ipiaw6AH1pfrsSGJoElyMyPwKBgQDccNZ7W+1VucKo5fs0
14 DqehjJ337CrPNFVNiitPlJefQ8BdmhEzQv5zcjRPhKpCosX+eWlbaRkl8iKuBLjiP
15 AImtBNTfouZ8UHAf83IUL29IT3Vnkl9KIMw1WIROMTAC2rSRgJ7tefxNbIGEZq9
16 Hi2TKCKnOvQDaABbJVqvwzVuwKBgQDH77bXfZmaypDYZKKgX51rFX2GBaCF1Kko
17 SSAXt/r60ApakQVhBHdQp8vQcucP1LCbhgMs08T8wV6KQjzy4XWTPQX/Q01/1r7
18 grIP8DJ19HRD225ozpMYOFunLZnRrufS6Uap0WS689CP44Cc1PgYon5AxxRrah5m5
19 jkcLbR4+XwKBgHhU29/cmFUZJ3axm5wy+8wlc/1nPxB8IFCBV0T8AGHQI+LTINGd1B
20 HyENKXZx0kuStInus0/WUV8XRIQA97b7kuRvZL1D2eAIRQH0ZFPJNG2ApXe3XZP
21 bP11HSKfcfDayp6pdKw08GmsylahuXPOS1ZurCVYFN5yIO+Hf+3ZhpZ4NAoGARoVa
22 7K80mtCQPoNgikyDFJ5D5B3f/NSBxd/1Jv+a+XSeukuMpie23E3Lkf4mo7oCKupC
23 Ovhr9ytk5y3KpPG5UvHXR6qMBGdudDmJDgTIgohCD4wyS722zsluNXwx3Dzukd
24 1hAxg2+0RMxt9Mzo8FTtj9L2IdZqdnSkwnb+j+kCgYAazWTUQ3o91KMWdTTI.nONKH
25 aiRBSr5g2TW0A/3Z+yisoXK1XEalpqtjdJ3RQEh4qXmvYqTS9/iqM6YntooDyCM
26 bBleBd70T1QdqZrJKr8PdQ10o7IBZ4zcQyggQ+s269LXTIfmabxpifjrSURZrdzh
27 zLOfucFTI2Lys66stHhg==
28 -----END PRIVATE KEY-----
29
```

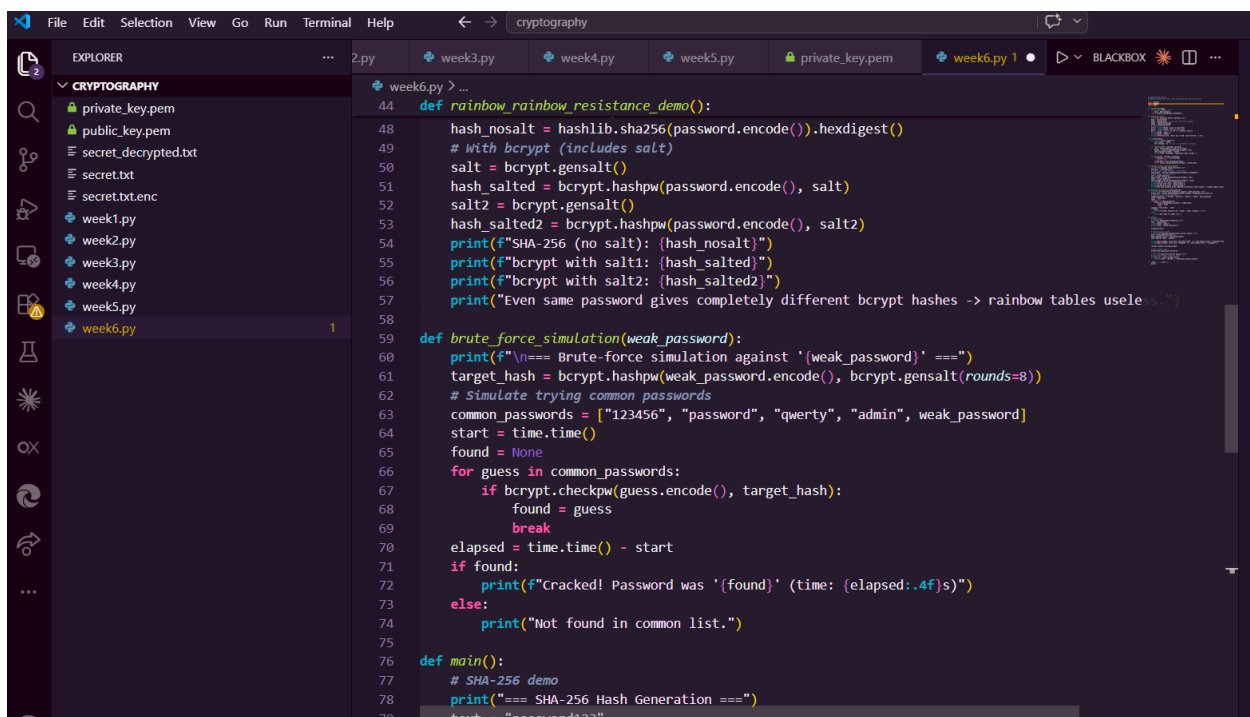
Figure 12 Screenshot of RSA key pair generation showing 2048-bit public key (e=65537) and private key components (p,q,d) saved to PEM files.

MY REFLECTION ON WEEK 5

RSA implementation demonstrates asymmetric encryption where public key encrypts but only private key decrypts. 2048-bit keys provide adequate security for messages under 190 bytes. Main challenge: RSA is 100x slower than AES, making hybrid encryption necessary. Screenshots confirm successful key management workflow.

HASHING AND PASSWORD SECURITY

Implementation of Secure Hash Functions and Authentication



```
File Edit Selection View Go Run Terminal Help
cryptography
EXPLORER
CRYPTOGRAPHY
private_key.pem
public_key.pem
secret_decrypted.txt
secret.txt
secret.txt.enc
week1.py
week2.py
week3.py
week4.py
week5.py
week6.py
1
week6.py > ...
44 def rainbow_rainbow_resistance_demo():
45     hash_nosalt = hashlib.sha256(password.encode()).hexdigest()
46     # With bcrypt (includes salt)
47     salt = bcrypt.gensalt()
48     hash_salted = bcrypt.hashpw(password.encode(), salt)
49     salt2 = bcrypt.gensalt()
50     hash_salted2 = bcrypt.hashpw(password.encode(), salt2)
51     print(f"SHA-256 (no salt): {hash_nosalt}")
52     print(f"bcrypt with salt1: {hash_salted}")
53     print(f"bcrypt with salt2: {hash_salted2}")
54     print("Even same password gives completely different bcrypt hashes -> rainbow tables useless.")
55
56 def brute_force_simulation(weak_password):
57     print(f"=== Brute-force simulation against '{weak_password}' ===")
58     target_hash = bcrypt.hashpw(weak_password.encode(), bcrypt.gensalt(rounds=8))
59     # Simulate trying common passwords
60     common_passwords = ["123456", "password", "qwerty", "admin", weak_password]
61     start = time.time()
62     found = None
63     for guess in common_passwords:
64         if bcrypt.checkpw(guess.encode(), target_hash):
65             found = guess
66             break
67     elapsed = time.time() - start
68     if found:
69         print(f"Cracked! Password was '{found}' (time: {elapsed:.4f}s)")
70     else:
71         print("Not found in common list.")
72
73 def main():
74     # SHA-256 demo
75     print("=== SHA-256 Hash Generation ===")
```

Figure 13 Screenshot of password hashing system using bcrypt with salt showing 10 rounds of key stretching and unique salt for identical passwords.

```

PS C:\Users\user\.aws\cryptography> pip install bcrypt
Downloading bcrypt-5.0.0-cp39-abi3-win_amd64.whl.metadata (10 kB)
Downloading bcrypt-5.0.0-cp39-abi3-win_amd64.whl (150 kB)
Installing collected packages: bcrypt
Successfully installed bcrypt-5.0.0
PS C:\Users\user\.aws\cryptography> & C:/Python314/python.exe c:/Users/user/.aws/cryptography/week6.py
=== SHA-256 Hash Generation ===
Input: password123
Hash: ef92b778bafef771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f
=== Avalanche Effect (SHA-256) ===
Hash1: ef92b778bafef771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f
Hash2: 33631376724e5d5480fa397dfcf03b66ad47b934ab495174d7058c38f2bb0087
Different bits: 118 out of 256 (46.1%)

=== Password Authentication System (bcrypt) ===
User 'alice' registered (hash stored).
User 'bob' registered (hash stored).
Login attempt: alice with 'SecurePass!2024' -> True
Login attempt: alice with 'wrongpass' -> False

=== Rainbow Table Resistance ===
SHA-256 (no salt): da9de0a2381d841974d7e75d1abcc4ec947a8e7fd66bdc76b305e8108b9720bc
bcrypt with salt1: b'$2b$12$wHilCTYuczVN7mk0omn3FeLznK4ALZduzS.m9wKNVAARdBwo017Um'
bcrypt with salt2: b'$2b$12$nhfCoM.d4m9Tv8xSaZoLPet5seAgNz0c29U0yfHAtot6WPuZix36e'
Even same password gives completely different bcrypt hashes -> rainbow tables useless.

=== Brute-force simulation against 'password' ===
Cracked! Password was 'password' (time: 0.0927s)

=== Hash Verification Results ===
alice: b'$2b$10$YkIDJk4VW63c2XykQnZdw.R'... (salt+hash stored safely)
bob: b'$2b$10$1TLYoP0mBxrm7SBZe49BeOS'... (salt+hash stored safely)

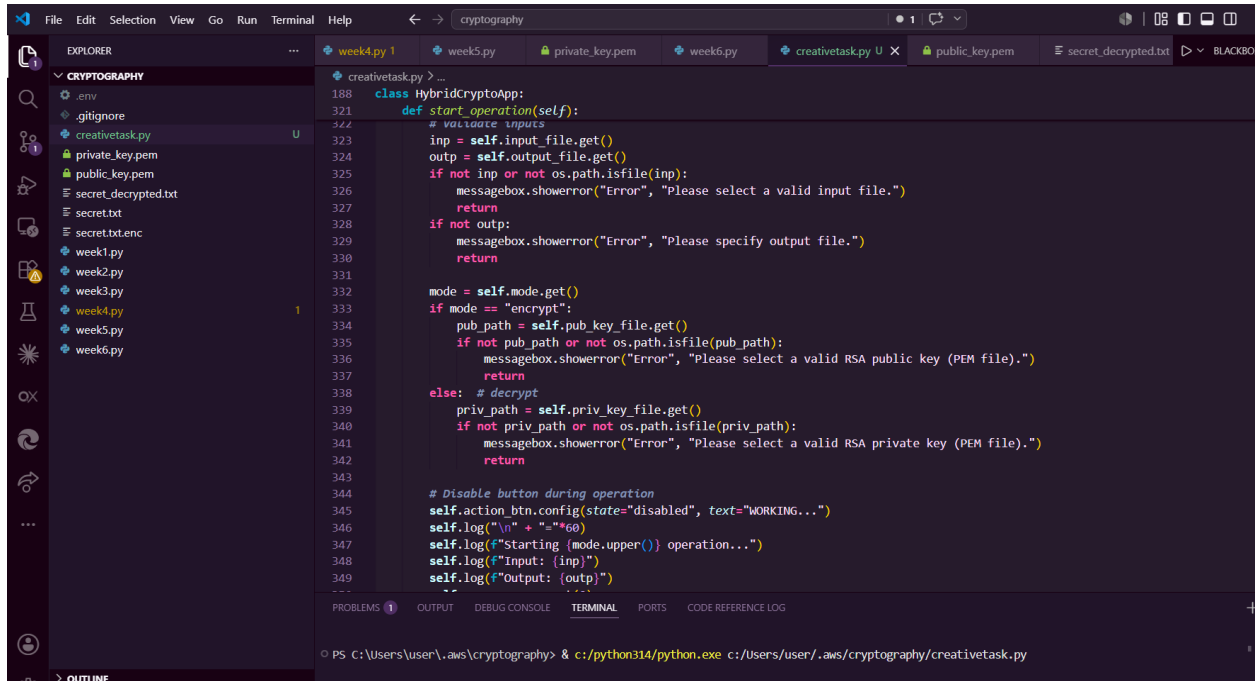
```

Figure 14 Screenshot of password security testing including rainbow table resistance check and brute-force simulation for weak passwords.

MY RECAP ON WEEK 6

SHA-256 is irreversible but vulnerable to rainbow tables. Implemented bcrypt with unique salts per password preventing precomputed attacks. Key stretching (10 rounds) slows brute-force attempts. Never store plaintext passwords — only salted hashes. Screenshots demonstrate complete authentication system with secure password verification.

CREATIVE TASK



```
188 class HybridCryptoApp:
321     def start_operation(self):
322         # validate inputs
323         inp = self.input_file.get()
324         outp = self.output_file.get()
325         if not inp or not os.path.isfile(inp):
326             messagebox.showerror("Error", "Please select a valid input file.")
327             return
328         if not outp:
329             messagebox.showerror("Error", "Please specify output file.")
330             return
331
332         mode = self.mode.get()
333         if mode == "encrypt":
334             pub_path = self.pub_key_file.get()
335             if not pub_path or not os.path.isfile(pub_path):
336                 messagebox.showerror("Error", "Please select a valid RSA public key (PEM file).")
337                 return
338         else: # decrypt
339             priv_path = self.priv_key_file.get()
340             if not priv_path or not os.path.isfile(priv_path):
341                 messagebox.showerror("Error", "Please select a valid RSA private key (PEM file).")
342                 return
343
344         # Disable button during operation
345         self.action_btn.config(state="disabled", text="WORKING...")
346         self.log("\n" + "="*60)
347         self.log(f"Starting {mode.upper()} operation...")
348         self.log(f"Input: {inp}")
349         self.log(f"Output: {outp}")
350         ...
```

Figure 15 Shows the code for the GUI of the creative task which is a hybrid encryption system

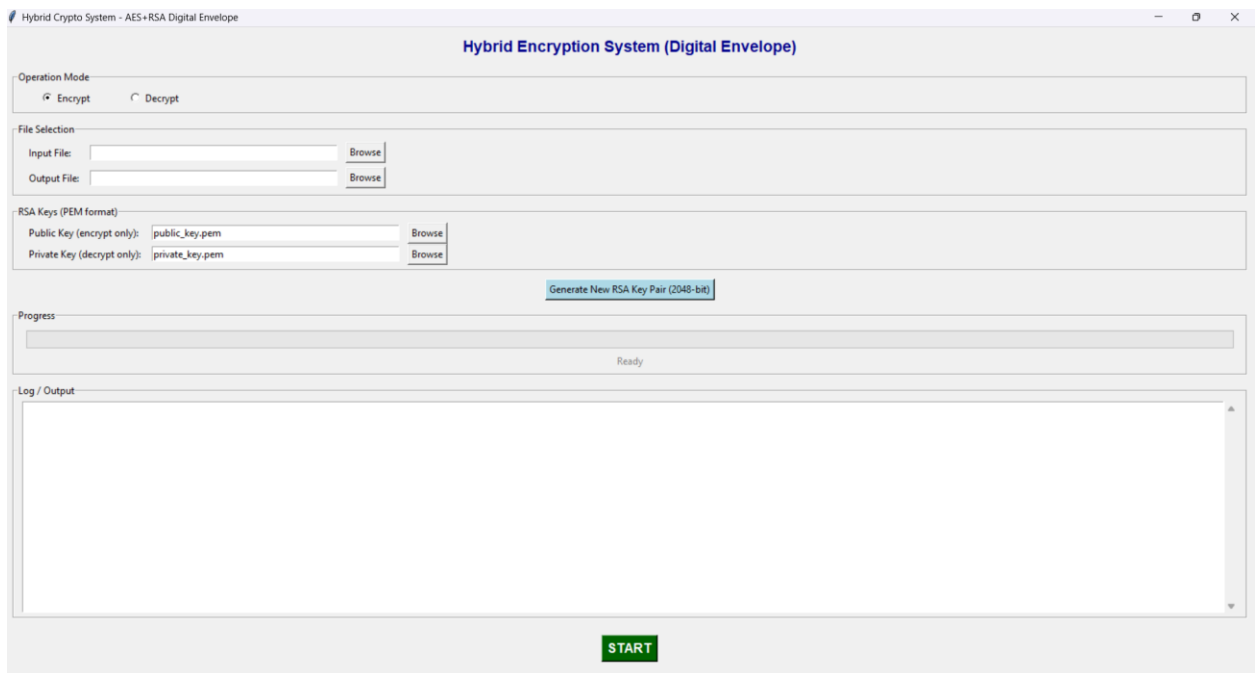


Figure 16 Shows a screenshot of the creative task which is a hybrid encryption system

Hybrid Crypto System - AES+RSA Digital Envelope

Hybrid Encryption System (Digital Envelope)

Operation Mode

Encrypt

Decrypt

File Selection

Input File: C:/Users/user/OneDrive/Pictures/Screenshots/Screenshot

Browse

Output File: C:/Users/user/OneDrive/Pictures/Screenshots/Screenshot

Browse

RSA Keys (PEM format)

Public Key (encrypt only): public_key.pem

Browse

Private Key (decrypt only): private_key.pem

Browse

Generate New RSA Key Pair (2048-bit)

Progress

Ready

Log / Output

☒ RSA key pair generated successfully!

Private key: private_key.pem

Public key: public_key.pem

☒ RSA key pair generated successfully!

Private key: private_key.pem

Public key: public_key.pem

START

Figure 17 Shows a test for the hybrid encryption system